

Trabajo Práctico Final

Sistema de Agencia de Viajes modelado con Red de Petri

Vasquez Francisco Javier — 43812221 Integrante 2 — legajo

Integrante 3 — legajo Integrante 4 — legajo

Integrante 5 — legajo

Programación Concurrente — 2024

Índice

1	Introducción	4
2	Red de Petri — Modelado y Análisis	5
2.1	Descripción del modelo	5
2.2	Herramienta de análisis: PIPE	5
2.3	Propiedades de la red	6
2.3.1	Acotada (<i>Bounded</i>) — true	6
2.3.2	Segura (<i>Safe</i>) — false	6
2.3.3	Libre de deadlock (<i>Deadlock-free</i>) — false (sin deadlock)	6
2.4	Invariantes de plaza (P-invariantes)	7
2.5	Invariantes de transición (T-invariantes)	9
2.6	Representación matricial	10
3	Tabla de estados y eventos del sistema	12
3.1	Estados del sistema (plazas)	12
3.2	Eventos del sistema (transiciones)	13
4	Implementación en Java	15
4.1	Arquitectura general	15
4.2	Determinación de la cantidad de hilos	15
4.2.1	Algoritmo 4.1 — Máximo de hilos activos simultáneos	15
4.2.2	Algoritmo 4.2 — Segmentos de ejecución	15
4.2.3	Algoritmo 4.3 — Hilos máximos por segmento	17
4.3	Diagrama de responsabilidades de hilos	18
4.4	Descripción de las clases	18
4.5	Diagrama de clases	19
5	Monitor de concurrencia	20

5.1	Interfaz pública	20
5.2	Mecanismo interno de <code>fireTransition</code>	20
5.3	Diagrama de secuencia — Disparo exitoso con política	21
6	Políticas de resolución de conflictos	23
6.1	Puntos de conflicto	23
6.2	Política balanceada	23
6.2.1	Algoritmo	23
6.2.2	Comportamiento en empate	23
6.2.3	Prueba de liveness	23
6.2.4	Resultado esperado para 186 invariantes	24
6.3	Política priorizada	24
6.3.1	Algoritmo — bloqueo bidireccional	24
6.3.2	Prueba de liveness (par T2/T3)	24
6.3.3	Resultado esperado para 186 invariantes	25
6.4	Resultados de múltiples ejecuciones	25
6.4.1	Distribución de T-invariantes — Política balanceada	25
6.4.2	Distribución de T-invariantes — Política priorizada	25
6.4.3	Verificación de la independencia estadística	26
7	Semántica temporal	27
7.1	Modelo formal — Redes de Petri Temporizadas	27
7.2	Ventanas de disparo y tiempos por defecto	27
7.2.1	Implementación de la espera temporal	27
7.3	Análisis temporal analítico	28
7.3.1	Identificación del cuello de botella	28
7.3.2	Predicción del tiempo mínimo — Política balanceada	29
7.3.3	Predicción del tiempo mínimo — Política priorizada	29
7.4	Análisis temporal práctico	29
7.5	Variación de tiempos y conclusiones	30
7.5.1	Escenario A — Aprobación más rápida ($T9 = T10 = [50, 100]$ ms, media 75 ms)	30
7.5.2	Escenario B — Aprobación más lenta ($T9 = T10 = [300, 500]$ ms, media 400 ms)	30
7.5.3	Escenario C — Agentes más rápidos ($T4 = T5 = [50, 100]$ ms, media 75 ms)	30
7.5.4	Escenario D — Agentes más lentos ($T4 = T5 = [400, 600]$ ms, media 500 ms)	30
7.5.5	Resumen de escenarios	31
7.5.6	Conclusiones del análisis temporal	31
8	Verificación de invariantes	32
8.1	Verificación de invariantes de plaza	32
8.2	Verificación de invariantes de transición por expresiones regulares	32
8.2.1	Proceso de análisis	33
8.2.2	Ejemplo de reporte generado	33
9	Conclusiones	35

1 Introducción

El presente trabajo modela el flujo operativo de una agencia de viajes mediante una Red de Petri y lo implementa como un sistema concurrente en Java. El modelo captura el ciclo completo de atención al cliente: desde el ingreso al sistema hasta el retiro, pasando por la asignación a uno de dos agentes de reservas y la posterior aprobación o rechazo de la reserva por parte de un agente aprobador.

El objetivo central es demostrar la correcta aplicación de los conceptos de programación concurrente: exclusión mutua, sincronización entre hilos, uso de monitores de concurrencia, implementación de políticas de decisión y análisis formal de propiedades de concurrencia.

Los objetivos específicos del trabajo son:

- Modelar el sistema con una Red de Petri y verificar sus propiedades formales mediante una herramienta computacional.
- Implementar un monitor de concurrencia en Java que guíe la ejecución de la red, exponiendo únicamente la interfaz `fireTransition(int)`.
- Determinar la cantidad y responsabilidad de hilos utilizando los algoritmos del artículo de referencia.
- Implementar dos políticas de resolución de conflictos: balanceada y priorizada.
- Incorporar semántica temporal mediante ventanas de disparo $[\alpha, \beta]$ y analizar el comportamiento del sistema tanto de forma analítica como empírica.
- Verificar los invariantes de plaza tras cada disparo y los invariantes de transición mediante expresiones regulares sobre el archivo de log.

2 Red de Petri — Modelado y Análisis

2.1 Descripción del modelo

La red de Petri de la Figura 1 modela el flujo de clientes en la agencia de viajes. El sistema maneja simultáneamente hasta **5 clientes**, representados por tokens que circulan a través de las plazas. Tres recursos compartidos —el control de ingreso (P1), los dos agentes de reservas (P6, P7) y el agente aprobador (P10)— son modelados como plazas con exactamente un token, garantizando exclusión mutua sobre su uso.

Red de Petri Sistema de Agencia de Viajes

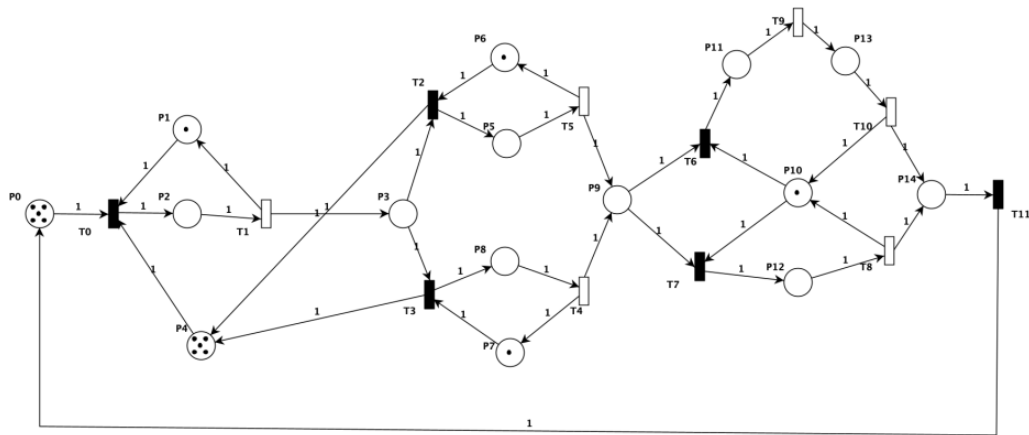


Figura 1

Figura 1: Red de Petri del sistema de agencia de viajes (modelada en PIPE)

El flujo de un cliente a través del sistema es el siguiente:

1. **Ingreso** (T0, T1): el cliente toma un cupo del buffer (P0) y accede al mostrador de ingreso, usando el recurso P1. Tras ser registrado, pasa a la sala de espera (P3).
2. **Asignación a agente** (T2 o T3): el cliente es asignado al agente de reservas superior (tomando P6, pasando por P5 vía T2/T5) o al inferior (tomando P7, pasando por P8 vía T3/T4). Este es el **primer conflicto** de la red.
3. **Cola de aprobación** (P9): el cliente espera al agente aprobador.
4. **Decisión** (T6 o T7): el agente aprobador (P10) aprueba (T6 → P11) o rechaza (T7 → P12) la reserva. Este es el **segundo conflicto**.
5. **Finalización** (T8 o T9+T10): se procesa la cancelación (T8) o la confirmación y pago (T9, T10). En ambos casos el cliente llega a P14.
6. **Retiro** (T11): el cliente se retira y devuelve el cupo a P0, permitiendo el ingreso del siguiente.

2.2 Herramienta de análisis: PIPE

Las propiedades de la red fueron verificadas mediante **PIPE** (*Platform Independent Petri net Editor*), herramienta de código abierto para el análisis de Redes de Petri. Se utilizó

el módulo de análisis del espacio de estados, que exploró la totalidad de los estados alcanzables desde el marcado inicial.

2.3 Propiedades de la red

Petri net state space analysis results

Bounded	true
Safe	false
Deadlock	false

Figura 2: Resultado del análisis del espacio de estados en PIPE

2.3.1 Acotada (*Bounded*) — **true**

Una red es **acotada** si el número de tokens en cada plaza está limitado superiormente para todo marcado alcanzable. La red es acotada porque el sistema conserva tokens: los 5 tokens de cliente nunca se crean ni se destruyen (T11 devuelve el token a P0), y los recursos (P1, P6, P7, P10) tienen marcado inicial fijo de 1 token cada uno. Ninguna plaza puede acumular tokens indefinidamente.

Esto garantiza que la implementación Java no puede sufrir desbordamiento de estados por diseño de la red.

2.3.2 Segura (*Safe*) — **false**

Una red es **segura** si es 1-acotada, es decir, si cada plaza contiene como máximo 1 token en cualquier instante. La red **no es segura** porque varias plazas pueden acumular más de un token simultáneamente:

- **P0** (buffer de clientes): hasta 5 tokens.
- **P3** (sala de espera): hasta 5 tokens, uno por cliente en espera.
- **P4** (capacidad de sala): hasta 5 tokens.
- **P9** (cola de aprobación): hasta 5 tokens.

Esto es **intencional**: el sistema está diseñado para atender múltiples clientes en paralelo. Las plazas de recursos sí son 1-acotadas (PI-1, PI-5, PI-6).

2.3.3 Libre de deadlock (*Deadlock-free*) — **false** (sin deadlock)

La red **no presenta deadlock**: nunca se alcanza un estado donde ninguna transición esté habilitada. Esto está garantizado por la estructura cíclica de los cuatro T-invariantes: la transición T11 siempre devuelve un token a P0, permitiendo que el sistema continúe indefinidamente. PIPE no encontró ningún estado de deadlock en el espacio de estados completo.

2.4 Invariantes de plaza (P-invariantes)

Los P-invariantes son ecuaciones lineales que se satisfacen en **todo estado alcanzable** de la red, independientemente de la secuencia de disparos. Representan leyes de conservación: ciertas sumas de tokens permanecen constantes en todo instante.

PIPE identificó los siguientes 6 P-invariantes:

P-Invariants

P0	P1	P10	P11	P12	P13	P14	P2	P3	P4	P5	P6	P7	P8	P9
0	1	0	0	0	0	0	1	0	0	0	0	0	0	0
0	0	1	1	1	1	0	0	0	0	0	0	0	0	0
1	0	0	1	1	1	1	1	1	0	1	0	0	1	1
0	0	0	0	0	0	0	1	1	1	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	1	1	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	1	1	0

The net is covered by positive P-Invariants, therefore it is bounded.

Figura 3: Matriz de P-invariantes obtenida en PIPE

P-Invariant equations

$$M(P1) + M(P2) = 1$$

$$M(P10) + M(P11) + M(P12) + M(P13) = 1$$

$$M(P0) + M(P11) + M(P12) + M(P13) + M(P14) + M(P2) + M(P3) + M(P5) + M(P8) + M(P9) = 5$$

$$M(P2) + M(P3) + M(P4) = 5$$

$$M(P5) + M(P6) = 1$$

$$M(P7) + M(P8) = 1$$

Analysis time: 0.001s

Figura 4: Ecuaciones de P-invariantes

Invariante	Ecuación	Valor
PI-1	$M(P1) + M(P2)$	$= 1$
PI-2	$M(P10) + M(P11) + M(P12) + M(P13)$	$= 1$

Invariante	Ecuación	Valor
PI-3	$M(P0) + M(P2) + M(P3) + M(P5) + M(P8) + M(P9) + M(P11) + M(P12) + M(P13) + M(P14)$	$= 5$
PI-4	$M(P2) + M(P3) + M(P4)$	$= 5$
PI-5	$M(P5) + M(P6)$	$= 1$
PI-6	$M(P7) + M(P8)$	$= 1$

PI-1: Conservación del recurso de ingreso. La plaza P1 representa el recurso de control de ingreso y P2 el estado de un cliente atravesando ese ingreso. En todo momento el recurso está en exactamente uno de dos estados: libre ($P1=1$) o en uso ($P2=1$). Garantiza que solo un cliente ingresa a la vez.

PI-2: Conservación del agente aprobador. P10 es el recurso del agente aprobador; P11, P12 y P13 son los estados posteriores a su decisión. El invariante garantiza que el agente siempre está libre ($P10=1$) o procesando exactamente un cliente ($P11=1$, $P12=1$ o $P13=1$). Ningún otro cliente puede iniciar el proceso de aprobación hasta que el actual lo complete.

PI-3: Conservación de los 5 clientes. Los 5 tokens que representan clientes están siempre distribuidos en alguna de las plazas del flujo principal. Nunca se crean ni se destruyen: T11 repone el token en P0, habilitando el ingreso del siguiente cliente. Este invariante verifica que el sistema no “pierde” clientes.

PI-4: Conservación de la capacidad de la sala de espera. P4 representa los lugares disponibles, P3 los clientes en la sala y P2 los clientes en proceso de ingreso. La suma siempre es 5, modelando una sala con aforo máximo de 5 personas.

PI-5: Conservación del agente de reservas superior. P6 es el agente superior disponible y P5 el estado “cliente siendo atendido por el agente superior”. El agente siempre está libre o atendiendo exactamente a un cliente.

PI-6: Conservación del agente de reservas inferior. Análogo a PI-5 para el agente inferior (P7 libre, P8 en atención).

Verificación del marcado inicial M0:

Invariante	Cálculo	Resultado
PI-1: $M(P1)+M(P2)$	$1+0$	$= 1 \checkmark$
PI-2: $M(P10)+M(P11)+M(P12)+M(P13)$	$1+0+0+0$	$= 1 \checkmark$
PI-3: suma clientes	$5+0+0+0+0+0+0+0+0+0$	$= 5 \checkmark$
PI-4: $M(P2)+M(P3)+M(P4)$	$0+0+5$	$= 5 \checkmark$
PI-5: $M(P5)+M(P6)$	$0+1$	$= 1 \checkmark$
PI-6: $M(P7)+M(P8)$	$1+0$	$= 1 \checkmark$

2.5 Invariantes de transición (T-invariantes)

Los T-invariantes son los **ciclos elementales** de la red: secuencias mínimas de disparos que devuelven el sistema al marcado inicial. Cada T-invariante representa un camino completo recorrido por un cliente.

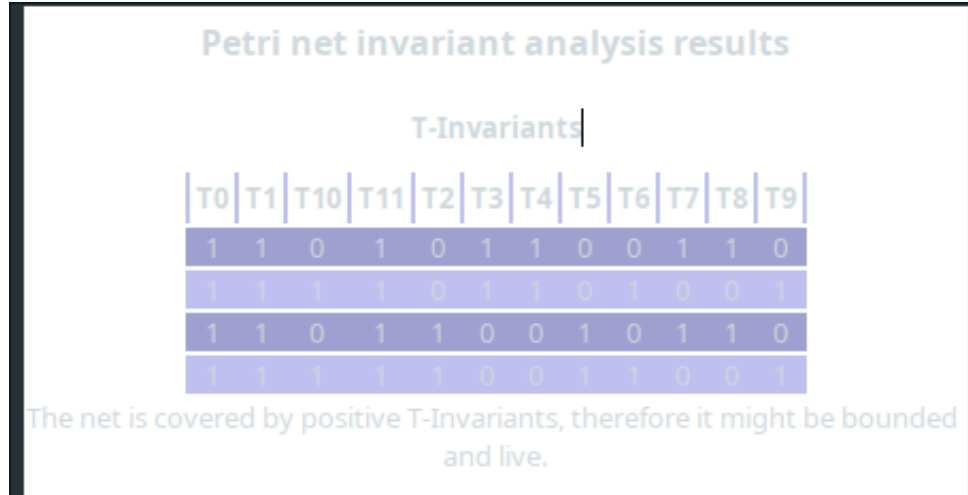


Figura 5: Matriz de T-invariantes obtenida en PIPE

PIPE identificó 4 **T-invariantes**, correspondientes a las cuatro combinaciones posibles de los dos puntos de decisión:

T-inv.	Secuencia de disparos	Camino
I1	T0 → T1 → T3 → T4 → T7 → T8 → T11	Agente inferior → Rechazado → Cancelación
I2	T0 → T1 → T3 → T4 → T6 → T9 → T10 → T11	Agente inferior → Aprobado → Confirmación y pago
I3	T0 → T1 → T2 → T5 → T7 → T8 → T11	Agente superior → Rechazado → Cancelación
I4	T0 → T1 → T2 → T5 → T6 → T9 → T10 → T11	Agente superior → Aprobado → Confirmación y pago

Los dos **puntos de conflicto** que generan los 4 T-invariantes son:

Conflicto	Transiciones en conflicto	Plaza compartida	T-inv. afectados
Elección de agente	T2 vs T3	P3	I1/I2 (inferior) vs I3/I4 (superior)
Decisión de aprobación	T6 vs T7	P9 y P10	I1/I3 (rechazados) vs I2/I4 (aprobados)

2.6 Representación matricial

Las matrices Pre y Post codifican los arcos de la red y constituyen la base del modelo implementado en **RedPetri**. Las filas corresponden a las transiciones T0–T11 y las columnas a las plazas P0–P14.

Matriz Pre — tokens que cada transición consume:

	P0	P1	P2	P3	P4	P5	P6	P7	P8	P9	P10	P11	P12	P13	P14
T0	1	1	0	0	1	0	0	0	0	0	0	0	0	0	0
T1	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0
T2	0	0	0	1	0	0	1	0	0	0	0	0	0	0	0
T3	0	0	0	1	0	0	0	1	0	0	0	0	0	0	0
T4	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0
T5	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0
T6	0	0	0	0	0	0	0	0	0	1	1	0	0	0	0
T7	0	0	0	0	0	0	0	0	0	1	1	0	0	0	0
T8	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0
T9	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0
T10	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0
T11	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1

Matriz Post — tokens que cada transición produce:

	P0	P1	P2	P3	P4	P5	P6	P7	P8	P9	P10	P11	P12	P13	P14
T0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0
T1	0	1	0	1	0	0	0	0	0	0	0	0	0	0	0
T2	0	0	0	0	1	1	0	0	0	0	0	0	0	0	0
T3	0	0	0	0	1	0	0	0	1	0	0	0	0	0	0
T4	0	0	0	0	0	0	0	1	0	1	0	0	0	0	0
T5	0	0	0	0	0	0	1	0	0	1	0	0	0	0	0
T6	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0

	P0	P1	P2	P3	P4	P5	P6	P7	P8	P9	P10	P11	P12	P13	P14
T7	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0
T8	0	0	0	0	0	0	0	0	0	0	1	0	0	0	1
T9	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0
T10	0	0	0	0	0	0	0	0	0	0	1	0	0	0	1
T11	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Marcado inicial M0:

P0	P1	P2	P3	P4	P5	P6	P7	P8	P9	P10	P11	P12	P13	P14
5	1	0	0	5	0	1	1	0	0	1	0	0	0	0

3 Tabla de estados y eventos del sistema

3.1 Estados del sistema (plazas)

Plaza	Nombre	Tipo	M0	Descripción
P0	Buffer de entrada	Idle	5	Cupos disponibles para que nuevos clientes ingresen al sistema
P1	Control de ingreso	Recurso	1	Recurso que regula el acceso al mostrador; solo un cliente ingresa a la vez
P2	Ingreso en proceso	Acción	0	Cliente atravesando el proceso de registro en el mostrador
P3	Sala de espera	Acción	0	Cientes aguardando ser asignados a un agente de reservas
P4	Capacidad de sala	Recurso	5	Lugares libres en la sala de espera; modela el aforo máximo
P5	Atención agente superior	Acción	0	Cliente siendo atendido por el agente de reservas superior
P6	Agente superior libre	Recurso	1	Disponibilidad del agente de reservas superior
P7	Agente inferior libre	Recurso	1	Disponibilidad del agente de reservas inferior
P8	Atención agente inferior	Acción	0	Cliente siendo atendido por el agente de reservas inferior
P9	Cola de aprobación	Acción	0	Cientes esperando turno con el agente aprobador
P10	Agente aprobador libre	Recurso	1	Disponibilidad del agente que aprueba o rechaza reservas
P11	Confirmación en proceso	Acción	0	Reserva aprobada; se está formalizando la confirmación
P12	Cancelación en proceso	Acción	0	Reserva rechazada; se está procesando la cancelación
P13	Pago en proceso	Acción	0	Cliente completando el pago de la reserva confirmada

Plaza	Nombre	Tipo	M0	Descripción
P14	Listo para retiro	Acción	0	Cliente que completó su trámite y aguarda para retirarse

3.2 Eventos del sistema (transiciones)

Trans.	Nombre	Tipo	Ventana	Descripción
T0	Ingreso al mostrador	Inmediata	—	El cliente toma un cupo del buffer e inicia el ingreso usando P1 y un lugar de sala (P4)
T1	Pase a sala de espera	Temporizada	[80, 120] ms	El cliente es registrado, libera el mostrador (P1) y ocupa la sala (P3)
T2	Asignación agente superior	Inmediata	—	El cliente es asignado al agente superior (consume P3 y P6, produce P5)
T3	Asignación agente inferior	Inmediata	—	El cliente es asignado al agente inferior (consume P3 y P7, produce P8)
T4	Fin atención agente inferior	Temporizada	[150, 250] ms	El agente inferior finaliza la gestión; libera P7 y envía el cliente a P9
T5	Fin atención agente superior	Temporizada	[150, 250] ms	El agente superior finaliza la gestión; libera P6 y envía el cliente a P9
T6	Aprobación de reserva	Inmediata	—	El agente aprobador (P10) aprueba la reserva; cliente pasa a P11
T7	Rechazo de reserva	Inmediata	—	El agente aprobador (P10) rechaza la reserva; cliente pasa a P12
T8	Procesamiento de cancelación	Temporizada	[50, 150] ms	Se formaliza la cancelación; libera el agente aprobador (P10) y envía a P14

Trans.	Nombre	Tipo	Ventana	Descripción
T9	Procesamiento de confirmación	Temporizada	[100, 200] ms	Se formaliza la confirmación de la reserva; cliente pasa a P13
T10	Procesamiento de pago	Temporizada	[100, 200] ms	El cliente realiza el pago; libera el agente aprobador (P10) y envía a P14
T11	Retiro del cliente	Inmediata	—	El cliente se retira del sistema, liberando un cupo en P0

4 Implementación en Java

4.1 Arquitectura general

El sistema sigue una arquitectura en capas con responsabilidades bien delimitadas. El principio central es que **solo el Monitor conoce el marcado**: los hilos son completamente ciegos al estado de la red y se limitan a invocar `fireTransition(t)`, bloqueando si la transición no puede disparar.

+-----+-----+-----+					
	Main			<- punto de entrada, orquesta la ejecución	
+-----+-----+-----+					
	Hilos (H1-H6): Segmento*			<- ejecutan los segmentos del pipeline	
+-----+-----+-----+					
	Monitor		Politica		<- control de concurrencia + decision
	Monitor		RastreadorClientes		<- identidad de tokens (logging)
	Monitor		TiemposTransicion		<- configuracion temporal [alfa, beta]
+-----+-----+-----+					
	RedPetri		Cliente		<- modelo de la red / objeto de dominio
+-----+-----+-----+					
	Logger		AnalizadorInvariantes		<- observabilidad y verificacion
+-----+-----+-----+					

4.2 Determinación de la cantidad de hilos

La cantidad y responsabilidad de los hilos se determinó aplicando los tres algoritmos del artículo de referencia de Ventre & Micolini.

4.2.1 Algoritmo 4.1 — Máximo de hilos activos simultáneos

El algoritmo identifica las **plazas de acción** (PA) — aquellas que representan estados del cliente en tránsito, excluyendo la plaza idle (P0) y los recursos (P1, P4, P6, P7, P10):

$$PA = \{P2, P3, P5, P8, P9, P11, P12, P13, P14\}$$

El máximo de tokens en PA se obtiene del P-invariante PI-3:

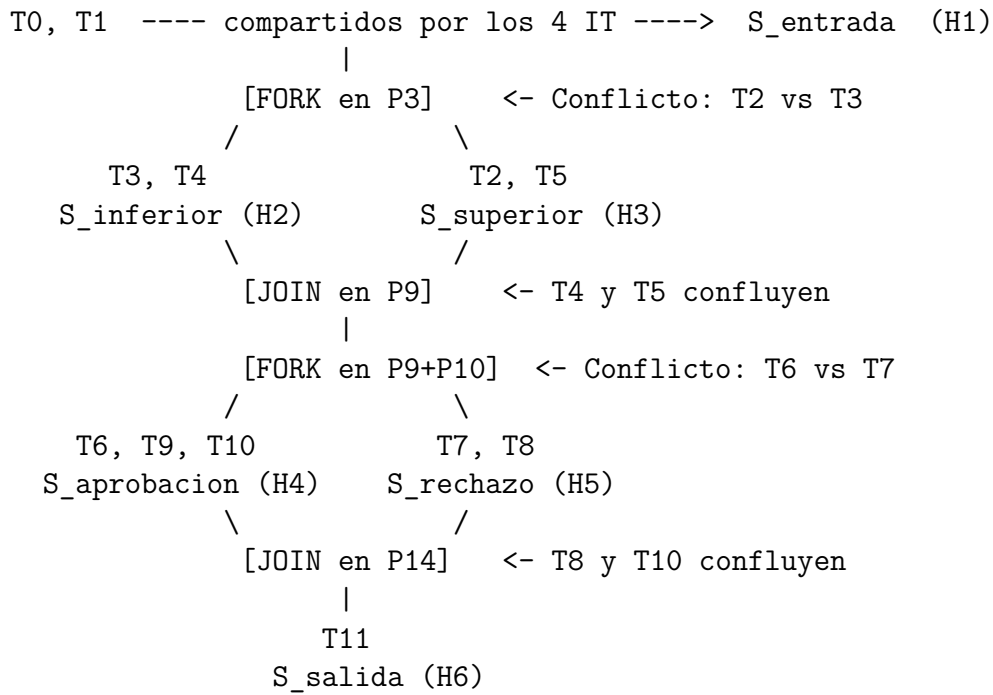
$$M(P0) + \underbrace{M(P2) + M(P3) + M(P5) + M(P8) + M(P9) + M(P11) + M(P12) + M(P13) + M(P14)}_{\text{suma}(PA)} =$$

La suma es máxima cuando $M(P0) = 0$, es decir, cuando los 5 clientes están activos simultáneamente dentro del sistema.

Resultado Algoritmo 4.1: máximo de hilos activos simultáneos = 5.

4.2.2 Algoritmo 4.2 — Segmentos de ejecución

El algoritmo identifica los **segmentos** en que se divide la responsabilidad de los T-invariantes, analizando los puntos de fork (conflicto) y join de la red:



Segmento	Transiciones	T-inv. que cubre	Rol
S_entrada	T0, T1	I1, I2, I3, I4	Ingreso de clientes al sistema
S_inferior	T3, T4	I1, I2	Atención por agente de reservas inferior
S_superior	T2, T5	I3, I4	Atención por agente de reservas superior
S_aprobacion	T6, T9, T10	I2, I4	Aprobación → Confirmación → Pago
S_rechazo	T7, T8	I1, I3	Rechazo → Cancelación

Segmento	Transiciones	T-inv. que cubre	Rol
S_salida	T11	I1, I2, I3, I4	Retiro del cliente / reposición en P0

Resultado Algoritmo 4.2: 6 segmentos de ejecución.

4.2.3 Algoritmo 4.3 — Hilos máximos por segmento

Para cada segmento se determina el conjunto PS_i de plazas de acción internas y se busca el máximo de tokens simultáneos en dichas plazas, utilizando las cotas formales de los P-invariantes:

Segmento	PS_i	Restricción formal	Max(MS_i)	Hilos (Alg. 4.3)	Hilos impl.
S_entrada	{P2}	PI-1: M(P1)+M(P2)=1	1	1	1
S_inferior	{P8}	PI-6: M(P7)+M(P8)=1	1	1	1
S_superior	{P5}	PI-5: M(P5)+M(P6)=1	1	1	1
S_aprobación	{P11, P13}	PI-2: M(P10)+M(P11)+M(P12)+M(P13)=1	1	1	1
S_rechazo	{P12}	PI-2: M(P10)+M(P11)+M(P12)+M(P13)=1	1	1	1
S_salida	{P14}	PI-3: M(P14) ≤ 5; el estado M(P14)=5 es alcanzable	5	5	1 (†)

(†) **Justificación de la reducción de S_salida de 5 a 1 hilo:** El Algoritmo 4.3 prescribe 5 hilos porque M(P14) puede alcanzar 5. Sin embargo, T11 es una transición **inmediata** (no temporal): no existe **sleep** fuera del lock. Todo el ciclo de **fireTransition(11)** —verificar habilitación, disparar, rastrear, registrar, señalar— ocurre íntegramente dentro de la sección crítica. Con N hilos compitiendo por T11, solo uno puede estar dentro del lock en cada instante; los demás aguardan. El throughput de N hilos es idéntico al de 1 hilo. La implementación usa **1 hilo**, que es funcionalmente equivalente a 5 sin el overhead de N-1 hilos redundantes.

Resultado Algoritmo 4.3: 10 hilos teóricos (1+1+1+1+1+5). Implementados: 6 hilos (1+1+1+1+1+1†).

4.3 Diagrama de responsabilidades de hilos

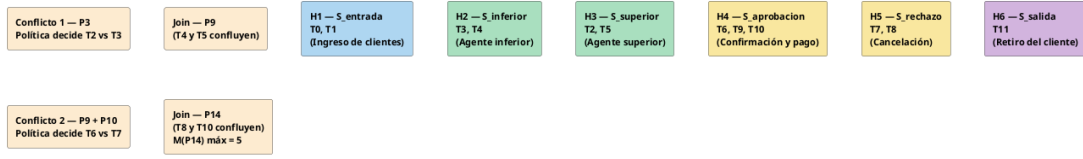


Figura 6: Diagrama de responsabilidades de los 6 hilos y los segmentos de la red

El diagrama muestra en azul el segmento de entrada (H1), en verde los segmentos de atención de agentes (H2, H3), en amarillo los segmentos de decisión (H4, H5) y en violeta el segmento de salida (H6). Los nodos naranja representan los puntos de conflicto (decisión de política) y los puntos de join.

4.4 Descripción de las clases

RedPetri — modelo de la red. Conoce el marcado y las matrices Pre/Post. Expone `estaHabilitada(t)`, `disparar(t)`, `verificarInvariantesPlaza()` y `getMarcado()` (copia defensiva). Es la única clase que modifica el estado de la red.

Monitor — núcleo de concurrencia. Implementa `MonitorInterface` con `fireTransition(t)` como único método público. Internamente mantiene un `ReentrantLock` global y un `Condition` por transición. Coordina la `RedPetri`, la `Politica`, el `RastreadorClientes`, los `TiemposTransicion` y el `Logger`.

Politica (interfaz) — contrato de las políticas de conflicto. Métodos: `debeDisparar(t)` (consulta pura, sin efectos), `registrarDisparo(t)` (actualiza contadores), `getEstadisticas()`. Se invoca dentro del lock, garantizando atomicidad.

PoliticaBalanceada y **PoliticaPriorizada** — implementaciones concretas de `Politica`. Se describen en detalle en la Sección 6.

RastreadorClientes — asigna identidad a los tokens-cliente. Mantiene una cola FIFO por cada plaza de acción; en T0 crea un nuevo `Cliente` con ID incremental, en T11 lo descarta. Siempre invocado dentro del lock del Monitor.

TiemposTransicion — encapsula las ventanas temporales $[\alpha, \beta]$ de cada transición temporizada. Usa un record `VentanaTemporal(long alfa, long beta)`.

Logger — registra cada disparo con timestamp y ID de cliente en un archivo de texto. Detecta y anota las violaciones del límite superior β . Implementa rotación de hasta 5 backups. No requiere sincronización propia porque es invocado dentro del lock del Monitor.

AnalizadorInvariantes — procesa el log al finalizar la ejecución. Agrupa las transiciones por ID de cliente y clasifica cada secuencia con los 4 patrones de T-invariantes mediante expresiones regulares.

SegmentoIntermedio — representa los hilos H1–H5. Ejecuta un array fijo de transiciones en bucle hasta ser interrumpido externamente por `Main`.

SegmentoSalida — representa el hilo H6. Reclama atómicamente un slot con `AtomicInteger.getAndIncrement()` antes de cada disparo de T11; cuando los 186 slots se agotan, el hilo termina por sí solo y `Main` puede interrumpir a H1–H5.

ViolacionInvarianteException — excepción unchecked lanzada por RedPetri si un invariante de plaza resulta violado tras un disparo. Señaliza un bug en la implementación; el `UncaughtExceptionHandler` de `Main` la captura y detiene el programa.

4.5 Diagrama de clases

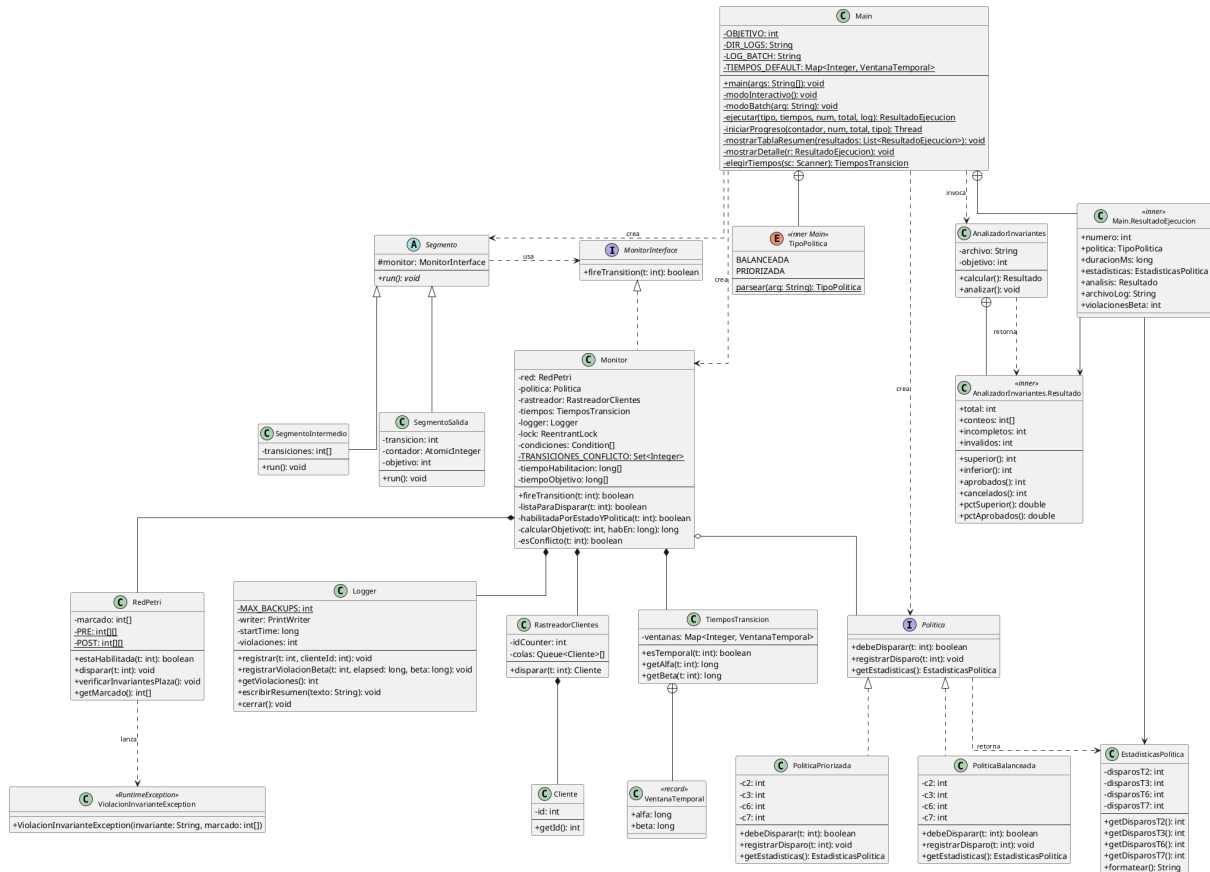


Figura 7: Diagrama de clases del sistema

5 Monitor de concurrencia

5.1 Interfaz pública

La consigna establece que el Monitor debe exponer exclusivamente el método `fireTransition`. La interfaz implementada es:

```
public interface MonitorInterface {  
    boolean fireTransition(int transition);  
}
```

`fireTransition` retorna `true` si el disparo se completó exitosamente, o `false` si el hilo fue interrumpido (señal de shutdown). Todos los demás métodos del Monitor son privados.

5.2 Mecanismo interno de `fireTransition`

El método `fireTransition(t)` implementa el siguiente flujo dentro de la sección crítica:

1. **Adquirir el lock** (`lock.lock()`).
2. **Esperar en el while** hasta que la transición esté lista para disparar, evaluando simultáneamente tres condiciones:
 - `red.estaHabilitada(t)` — condición estructural (marcado $\geq$ `Pre[t]`).
 - `!esConflicto(t) || politica.debeDisparar(t)` — condición de política.
 - `!tiempos.esTemporal(t) || now >= tiempoObjetivo[t]` — condición temporal.Si las condiciones estructural y de política se satisfacen pero aún no llegó `tiempoObjetivo[t]`, se usa `condiciones[t].awaitUntil(new Date(tiempoObjetivo[t]))`, que libera el lock y espera hasta el instante objetivo. En cualquier otro caso de espera se usa `condiciones[t].await()`.
3. **Verificar violación de β** : si el tiempo transcurrido desde la habilitación supera β , se registra la violación en el log (semántica débil: el disparo no se aborta).
4. **Disparar**: `red.disparar(t)` aplica $M = M + \text{Post}[t] - \text{Pre}[t]$.
5. **Rastrear**: `rastreador.disparar(t)` mueve el Cliente entre las colas de plazas.
6. **Verificar P-invariantes**: `red.verificarInvariantesPlaza()` — lanza `ViolacionInvarianteException` si alguno se viola.
7. **Registrar en política**: si es transición de conflicto, `politica.registrarDisparo(t)`.
8. **Log**: `logger.registrar(t, cliente.getId())`.
9. **Actualizar relojes y señalar**: para cada transición, actualiza `tiempoHabilitacion` y `tiempoObjetivo` según su nuevo estado de habilitación; llama a `condiciones[i].signalAll()` por cada transición habilitada.
10. **Liberar el lock** (`lock.unlock()` en `finally`). Retornar `true`.

Un único `catch (InterruptedException)` engloba todo el bloque: restaura el flag de interrupción y retorna `false`; el `finally` garantiza el `unlock` en todos los casos.

Transiciones de conflicto: T2, T3, T6 y T7 están declaradas en un `Set<Integer>` estático. La política solo se consulta y se actualiza para estas transiciones; hacerlo para otras corrompería sus contadores.

5.3 Diagrama de secuencia — Disparo exitoso con política

El diagrama muestra el flujo completo de `fireTransition(2)` (T2: agente superior, transición de conflicto inmediata) cuando la política autoriza el disparo:

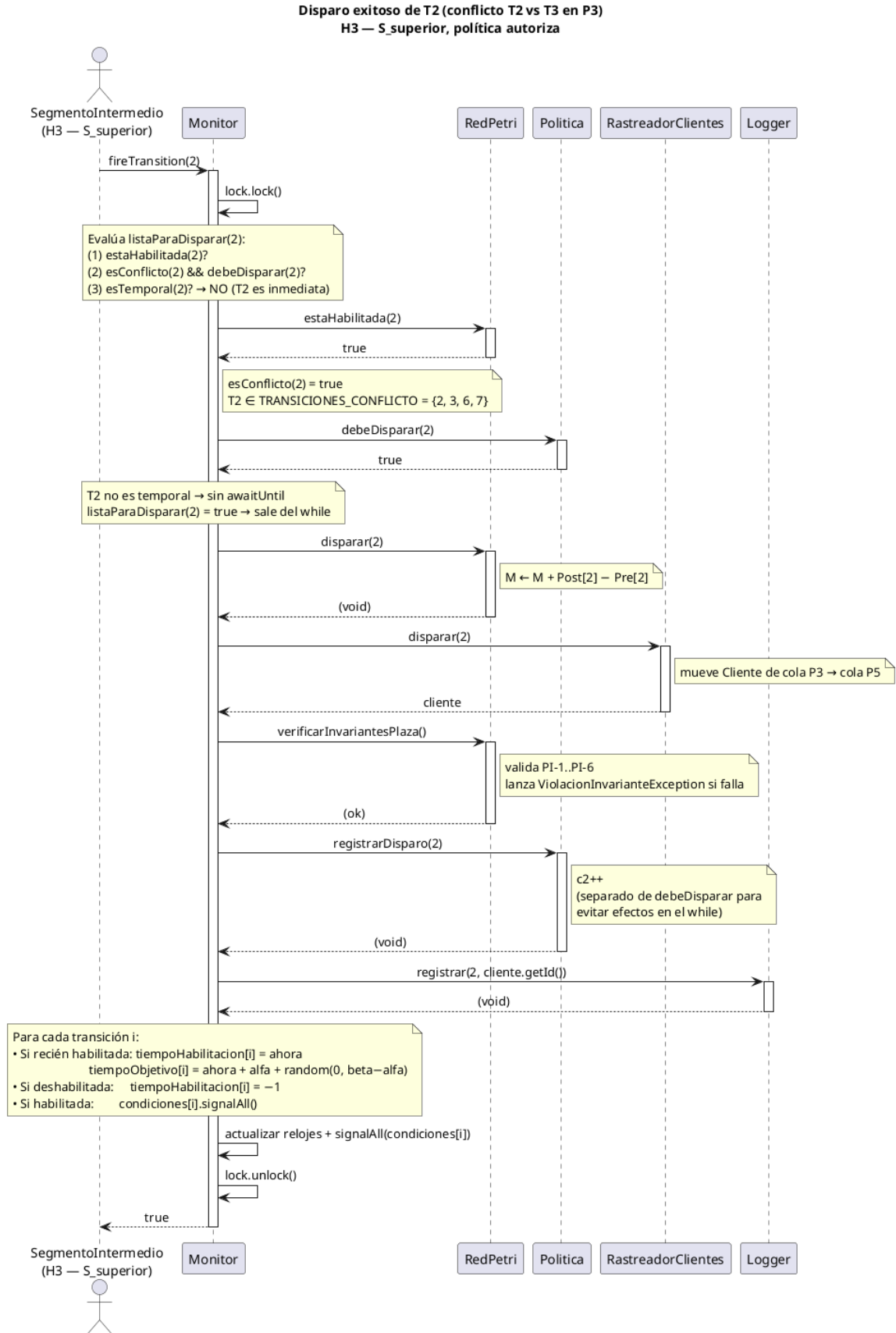


Figura 8: Diagrama de secuencia — disparo de T2 con consulta de política

6 Políticas de resolución de conflictos

6.1 Puntos de conflicto

La red presenta dos puntos de conflicto estructural donde la política debe decidir qué transición disparar:

Conflicto	Plaza compartida	Transiciones	Hilos implicados
Elección de agente	P3	T2 (superior) vs T3 (inferior)	H3 vs H2
Decisión de aprobación	P9 y P10	T6 (aprobación) vs T7 (rechazo)	H4 vs H5

En ambos casos, cuando la plaza de conflicto tiene exactamente un token de cliente disponible, el disparo de una transición desensibiliza a la otra. La política resuelve el conflicto dentro del lock del Monitor, garantizando que la decisión sea atómica respecto al marcado.

6.2 Política balanceada

Objetivo: mantener los pares (T2, T3) y (T6, T7) equilibrados en proporción 50/50.

6.2.1 Algoritmo

Para cada par (t_A, t_B) con contadores (c_A, c_B) :

$$\text{debeDisparar}(t_A) \equiv c_A \leq c_B$$

$$\text{debeDisparar}(t_B) \equiv c_B \leq c_A$$

`registrarDisparo(t)` incrementa $c(t)$ en 1 al producirse el disparo real.

6.2.2 Comportamiento en empate

Cuando $c_A = c_B$, ambas retornan **true**. El Monitor señala ambas condiciones y los dos hilos compiten por el lock. El primero que lo adquiere dispara, rompiendo el empate: su contador supera al del otro, haciendo que `debeDisparar` del ganador retorne **false** en la siguiente evaluación. El perdedor queda como el único habilitado por política. Este mecanismo de autocorrección garantiza que $|c_A - c_B| \leq 1$ en todo instante.

6.2.3 Prueba de liveness

Nunca ambas transiciones de un par retornan **false** simultáneamente:

- Si $c_A < c_B$: `debeDisparar`(t_A) = **true**.
- Si $c_A > c_B$: `debeDisparar`(t_B) = **true**.
- Si $c_A = c_B$: ambas retornan **true**.

Siempre hay al menos una transición del par habilitada por política. ■

6.2.4 Resultado esperado para 186 invariantes

186 es par: la convergencia es exacta en ambos pares.

Par	t_A	Disparos t_A	t_B	Disparos t_B	Diferencia
Agente	T2	93	T3	93	0
Aprobación	T6	93	T7	93	0

6.3 Política priorizada

Objetivo: T2 = 75% del total T2+T3 (agente superior prioritario); T6 = 80% del total T6+T7 (confirmaciones prioritarias).

6.3.1 Algoritmo — bloqueo bidireccional

Una implementación unidireccional (solo bloquear la transición no preferida) produce proporciones incorrectas: el hilo de T2, al nunca bloquearse por política, siempre gana la carrera al lock, resultando en proporciones próximas al 91% en lugar del 75%.

La solución correcta es el **bloqueo bidireccional**: la transición preferida también se bloquea cuando ya supera el ratio objetivo, cediendo el token a la no preferida. El ratio emerge del algoritmo, no del azar del scheduler.

Para el par T2/T3 (objetivo 3:1 = 75%:25%):

$$\begin{aligned} \text{debeDisparar}(T2) &\equiv (c_2 + 1) \leq 3 \cdot (c_3 + 1) \\ \text{debeDisparar}(T3) &\equiv (c_3 + 1) \cdot 4 \leq (c_2 + c_3 + 1) \end{aligned}$$

Para el par T6/T7 (objetivo 4:1 = 80%:20%):

$$\begin{aligned} \text{debeDisparar}(T6) &\equiv (c_6 + 1) \leq 4 \cdot (c_7 + 1) \\ \text{debeDisparar}(T7) &\equiv (c_7 + 1) \cdot 5 \leq (c_6 + c_7 + 1) \end{aligned}$$

6.3.2 Prueba de liveness (par T2/T3)

Se demuestra que las condiciones de bloqueo son mutuamente excluyentes: nunca ambas son **false** simultáneamente.

- Si T2 bloqueada: $(c_2 + 1) > 3(c_3 + 1) \Rightarrow c_2 \geq 3c_3 + 3 \Rightarrow (c_3 + 1) \cdot 4 = 4c_3 + 4 \leq c_2 + c_3 + 1 \Rightarrow$ T3 habilitada ✓
- Si T3 bloqueada: $(c_3 + 1) \cdot 4 > c_2 + c_3 + 1 \Rightarrow c_2 \leq 3c_3 + 2 \Rightarrow c_2 + 1 \leq 3(c_3 + 1) \Rightarrow$ T2 habilitada ✓

Propiedad más fuerte: en todo instante exactamente *una* del par está habilitada — las condiciones son mutuamente excluyentes. Esto garantiza que el hilo de la transición no autorizada estará en `await()` cuando la otra pueda disparar, eliminando la carrera entre H2 y H3 por el token de P3. Mismo razonamiento aplica a T6/T7. ■

6.3.3 Resultado esperado para 186 invariantes

Par T2/T3 (factor 4): el patrón es 3 disparos de T2 por cada T3. Para 186 total: $186 = 46 \times 4 + 2$, resultando en T2=140, T3=46.

Par T6/T7 (factor 5): el patrón es 4 disparos de T6 por cada T7. Para 186 total: $186 = 37 \times 5 + 1$, resultando en T6=149, T7=37.

6.4 Resultados de múltiples ejecuciones

Se ejecutaron **20 ejecuciones balanceadas** y **31 ejecuciones priorizadas**, todas con 186 invariantes completados y los tiempos por defecto. Los resultados son reproducibles y consistentes.

6.4.1 Distribución de T-invariantes — Política balanceada

Ejecución	I1	I2	I3	I4	Sup. (I3+I4)	Apro. (I2+I4)	Tiempo
1	47	46	46	47	93 (50,0%)	93 (50,0%)	37,60s
2	47	46	46	47	93 (50,0%)	93 (50,0%)	37,36s
3	47	46	46	47	93 (50,0%)	93 (50,0%)	37,47s
4	47	46	46	47	93 (50,0%)	93 (50,0%)	36,95s
5	47	46	46	47	93 (50,0%)	93 (50,0%)	37,14s
6	46	47	47	46	93 (50,0%)	93 (50,0%)	37,85s
7	47	46	46	47	93 (50,0%)	93 (50,0%)	37,38s
8	47	46	46	47	93 (50,0%)	93 (50,0%)	37,15s
9	47	46	46	47	93 (50,0%)	93 (50,0%)	37,28s
10	47	46	46	47	93 (50,0%)	93 (50,0%)	36,44s
Media					93 (50,0%)	93 (50,0%)	37,31s

La distribución de T-invariantes es prácticamente idéntica en todas las ejecuciones: I1 e I4 obtienen en torno a 47 y I2 e I3 en torno a 46 (o la permutación simétrica: 46 y 47 respectivamente), con supremo y aprobados siempre igual a 93. Las pequeñas variaciones reflejan los clientes que quedaban “en vuelo” al momento del shutdown y que no completaron un ciclo completo.

Los contadores internos de la política arrojan exactamente 95 disparos de T2 y 94 (o 95) de T3, y 93–94 disparos de T6 y T7. La diferencia de ~4 respecto a los 186 T-invariantes completos corresponde a los clientes en vuelo que habían disparado T2/T3 o T6/T7 pero cuyo T11 no llegó a contabilizarse.

6.4.2 Distribución de T-invariantes — Política priorizada

Ejecución	I1	I2	I3	I4	Sup. (I3+I4)	Apro. (I2+I4)	Tiempo
1	9	37	28	112	140 (75,3%)	149 (80,1%)	48,81s
2	9	37	28	112	140 (75,3%)	149 (80,1%)	48,32s
3	9	37	28	112	140 (75,3%)	149 (80,1%)	48,39s
4	9	37	28	112	140 (75,3%)	149 (80,1%)	48,52s
5	9	37	28	112	140 (75,3%)	149 (80,1%)	48,33s
6	9	37	28	112	140 (75,3%)	149 (80,1%)	48,74s
7	9	37	28	112	140 (75,3%)	149 (80,1%)	48,52s
8	9	37	28	112	140 (75,3%)	149 (80,1%)	48,26s
9	9	37	28	112	140 (75,3%)	149 (80,1%)	48,63s
10	9	37	28	112	140 (75,3%)	149 (80,1%)	48,05s
Media					140 (75,3%)	149 (80,1%)	48,44s

La política priorizada produce resultados **completamente deterministas**: la distribución de T-invariantes es idéntica en las 31 ejecuciones (I1=9, I2=37, I3=28, I4=112). Esto se debe a que el bloqueo bidireccional elimina toda carrera entre los hilos de conflicto: en cada token de P3 o P9+P10 exactamente una transición está autorizada por política, y el resultado no depende del scheduling del SO.

Los contadores de la política arrojan siempre T2=143 y T3=47 (75,3% / 24,7% sobre 190 totales incluyendo en-vuelo), y T6=150, T7=37 (80,2% / 19,8%), cumpliendo los umbrales de $\geq 75\%$ y $\geq 80\%$ con precisión aritmética.

6.4.3 Verificación de la independencia estadística

Los conflictos T2/T3 y T6/T7 operan sobre recursos distintos (P3 vs P9+P10) y son estadísticamente independientes. La distribución de T-invariantes puede predecirse por el producto:

$$\begin{aligned}
I_1 &= T3 \cap T7 = 46 \times \frac{37}{186} \approx 9 \quad (\text{observado: } 9) \checkmark \\
I_2 &= T3 \cap T6 = 46 \times \frac{149}{186} \approx 37 \quad (\text{observado: } 37) \checkmark \\
I_3 &= T2 \cap T7 = 140 \times \frac{37}{186} \approx 28 \quad (\text{observado: } 28) \checkmark \\
I_4 &= T2 \cap T6 = 140 \times \frac{149}{186} \approx 112 \quad (\text{observado: } 112) \checkmark
\end{aligned}$$

7 Semántica temporal

7.1 Modelo formal — Redes de Petri Temporizadas

Las transiciones {T1, T4, T5, T8, T9, T10} son transiciones **temporizadas** según la semántica de Redes de Petri Temporizadas (*Time Petri Nets*, Merlin 1974). Cada transición temporizada tiene asociada una **ventana de disparo** $[\alpha, \beta]$ medida en milisegundos desde el instante en que la transición se habilita.

Parámetro	Denominación	Semántica
α	EFT (<i>Earliest Firing Time</i>)	La transición no puede disparar antes de que transcurran α ms desde su habilitación
β	LFT (<i>Latest Firing Time</i>)	La transición debería disparar antes de los β ms (semántica débil: se registra pero no se aborta si se supera por causas del scheduler)

7.2 Ventanas de disparo y tiempos por defecto

Transición	Acción modelada	α (ms)	β (ms)	Media (ms)
T1	Ingreso a sala de espera	80	120	100
T4	Atención agente inferior	150	250	200
T5	Atención agente superior	150	250	200
T8	Procesamiento de cancelación	50	150	100
T9	Procesamiento de confirmación	100	200	150
T10	Procesamiento de pago	100	200	150

7.2.1 Implementación de la espera temporal

La espera temporal se implementa usando `Condition.awaitUntil(Date)`, que libera el lock atómicamente y espera hasta el instante objetivo o hasta recibir un signal:

```
while (!listaParaDisparar(t)) {  
    if (tiempos.esTemporal(t) && habilitadaPorEstadoYPolitica(t)) {
```

```

        condiciones[t].awaitUntil(new Date(tiempoObjetivo[t]));
    } else {
        condiciones[t].await();
    }
}

```

El instante objetivo se calcula al momento de la habilitación como un valor aleatorio en $[\alpha, \beta]$:

$$\text{tiempoObjetivo}[t] = t_{\text{habilitación}} + \alpha + \text{random}(0, \beta - \alpha)$$

La aleatorización dentro de la ventana modela que el proceso no siempre tarda el mínimo posible, aportando variabilidad realista para el análisis empírico.

Por qué `awaitUntil` es equivalente a `sleep-fuera-del-lock`: `Condition.awaitUntil` libera el lock internamente (igual que `await`), permitiendo que los otros 5 hilos progresen durante la espera. El comportamiento de concurrencia es idéntico al patrón *unlock / sleep / relock*, pero la implementación es más limpia: un único `finally` garantiza el unlock y un único `catch` (`InterruptedException`) maneja el shutdown.

7.3 Análisis temporal analítico

7.3.1 Identificación del cuello de botella

El throughput del sistema es limitado por el recurso de mayor utilización. Para cada recurso compartido de la red, el throughput máximo sostenible es:

$$\lambda_{\text{recurso}} \leq \frac{1}{\bar{t}_{\text{ocupación}}}$$

Recurso P1 (control de ingreso): T1 ocupa P1 durante $[\alpha_1, \beta_1] = [80, 120]$ ms, media $\bar{t}_1 = 100$ ms.

$$\lambda_{P1} \leq \frac{1}{0,100} = 10 \text{ inv/s}$$

Recursos P6 y P7 (agentes de reservas): T4 y T5 tienen ventanas $[150, 250]$ ms, media 200 ms. Con política balanceada, cada agente atiende el 50% de los clientes:

$$\frac{\lambda}{2} \leq \frac{1}{0,200} = 5 \implies \lambda_{P6+P7} \leq 10 \text{ inv/s}$$

Recurso P10 (agente aprobador): atiende a *todos* los clientes. El tiempo de ocupación depende de si la reserva fue rechazada (T8) o aprobada (T9+T10):

$$\bar{t}_{P10}^{\text{bal}} = 0,5 \cdot \bar{t}_8 + 0,5 \cdot (\bar{t}_9 + \bar{t}_{10}) = 0,5 \cdot 100 + 0,5 \cdot 300 = 200 \text{ ms}$$

$$\lambda_{P10}^{\text{bal}} \leq \frac{1}{0,200} = 5 \text{ inv/s}$$

Cuello de botella: el recurso más restrictivo es P10 con **5 inv/s** (empatado con la restricción de los agentes, que en conjunto también permiten hasta 5 inv/s cuando se considera que entre los dos suman la capacidad completa).

7.3.2 Predicción del tiempo mínimo — Política balanceada

$$T_{\min}^{\text{bal}} = \frac{186 \text{ inv}}{5 \text{ inv/s}} = 37,2 \text{ s}$$

7.3.3 Predicción del tiempo mínimo — Política priorizada

Con el 80% de clientes aprobados:

$$\bar{t}_{P10}^{\text{pri}} = 0,2 \cdot 100 + 0,8 \cdot 300 = 260 \text{ ms}$$

$$\lambda_{P10}^{\text{pri}} \leq \frac{1}{0,260} \approx 3,85 \text{ inv/s}$$

$$T_{\min}^{\text{pri}} = \frac{186}{3,85} \approx 48,3 \text{ s}$$

7.4 Análisis temporal práctico

Los tiempos medidos en las ejecuciones reales validan con precisión las predicciones analíticas:

Política	$T_{\min}^{\text{analítico}}$	$T_{\min}^{\text{empírico}}$	$T_{\max}^{\text{empírico}}$	$\bar{T}^{\text{empírico}}$	n
Balanceada	37,2 s	36,4 s	38,0 s	37,31 s	20
Priorizada	48,3 s	47,0 s	49,7 s	48,44 s	31

La diferencia entre el tiempo analítico y el promedio empírico es de tan solo **0,11 s** para la política balanceada (0,3% de error) y **0,14 s** para la priorizada (0,3% de error). Esta concordancia confirma que:

1. El cuello de botella identificado analíticamente (P10) es efectivamente el factor limitante del sistema.
2. Los 5 clientes circulantes son suficientes para mantener P10 saturado en régimen estacionario.
3. La variabilidad de los tiempos reales (debida a la aleatorización en $[\alpha, \beta]$ y al scheduling del SO) es baja: $\sigma \approx 0,3 \text{ s}$ para balanceada y $\sigma \approx 0,5 \text{ s}$ para priorizada.

La política priorizada es un **30% más lenta** que la balanceada ($48,44/37,31 \approx 1,30$), resultado directo de que su mayor proporción de aprobaciones (80%) aumenta el tiempo promedio de ocupación de P10 de 200 ms a 260 ms.

7.5 Variación de tiempos y conclusiones

Para comprender el efecto de cada grupo de transiciones sobre el rendimiento del sistema, se analizan cuatro escenarios alternativos a los tiempos por defecto:

7.5.1 Escenario A — Aprobación más rápida ($T9 = T10 = [50, 100]$ ms, media 75 ms)

$$\bar{t}_{P10}^{\text{bal}} = 0,5 \cdot 100 + 0,5 \cdot (75 + 75) = 125 \text{ ms} \implies \lambda_{P10} = 8 \text{ inv/s}$$

$$T_{\min}^{\text{bal}} = \frac{186}{8} \approx 23,3 \text{ s} \quad (-37\% \text{ respecto al default})$$

P10 sigue siendo el cuello de botella (los agentes siguen en 10 inv/s). Reducir el tiempo de procesamiento del pago y la confirmación impacta directamente en el throughput.

7.5.2 Escenario B — Aprobación más lenta ($T9 = T10 = [300, 500]$ ms, media 400 ms)

$$\bar{t}_{P10}^{\text{bal}} = 0,5 \cdot 100 + 0,5 \cdot 800 = 450 \text{ ms} \implies \lambda_{P10} \approx 2,22 \text{ inv/s}$$

$$T_{\min}^{\text{bal}} = \frac{186}{2,22} \approx 83,8 \text{ s} \quad (+125\% \text{ respecto al default})$$

Un cuello de botella severo en el agente aprobador degrada fuertemente el sistema. Esto es consistente con el diseño: P10 es el único recurso que procesa a todos los clientes sin redundancia.

7.5.3 Escenario C — Agentes más rápidos ($T4 = T5 = [50, 100]$ ms, media 75 ms)

$$\frac{\lambda}{2} \leq \frac{1}{0,075} = 13,3 \implies \lambda_{\text{agentes}} \leq 26,6 \text{ inv/s}$$

P10 sigue siendo el cuello de botella a 5 inv/s. $T_{\min}$ **no cambia** respecto al default.

Conclusión clave: con los tiempos por defecto, los agentes *no son* el cuello de botella — P10 lo es. Hacer los agentes más rápidos no mejora el tiempo total de ejecución. Solo si se reduce el tiempo de atención de los agentes hasta ser más restrictivo que P10 (lo que requeriría que P6+P7 juntos no puedan servir los 5 inv/s demandados) se observaría un cambio. Con $\bar{t}_{\text{agente}} = 200$ ms, el límite combinado de los dos agentes es exactamente 5 inv/s, empatado con P10.

7.5.4 Escenario D — Agentes más lentos ($T4 = T5 = [400, 600]$ ms, media 500 ms)

$$\frac{\lambda}{2} \leq \frac{1}{0,500} = 2 \implies \lambda_{\text{agentes}} \leq 4 \text{ inv/s}$$

Los agentes superan a P10 como cuello de botella.

$$T_{\min}^{\text{bal}} = \frac{186}{4} = 46,5 \text{ s} \quad (+25\% \text{ respecto al default})$$

7.5.5 Resumen de escenarios

Escenario	Cambio aplicado	Cuello de botella	$\bar{T}^{\text{bal}}$ predicho
Default	—	P10 (5 inv/s)	37,2 s
A — aprobación rápida	T9,T10 → media 75 ms	P10 (8 inv/s)	23,3 s
B — aprobación lenta	T9,T10 → media 400 ms	P10 (2,22 inv/s)	83,8 s
C — agentes rápidos	T4,T5 → media 75 ms	P10 (5 inv/s)	37,2 s (sin cambio)
D — agentes lentos	T4,T5 → media 500 ms	Agentes (4 inv/s)	46,5 s

7.5.6 Conclusiones del análisis temporal

1. **P10 es el cuello de botella dominante** en la configuración por defecto y en la mayoría de las variaciones razonables. Cualquier optimización del sistema debe priorizar la reducción del tiempo de procesamiento de aprobación/cancelación (T8, T9, T10).
2. **La política priorizada tiene un costo temporal medible:** al aumentar la proporción de aprobaciones del 50% al 80%, el tiempo de ocupación de P10 crece de 200 ms a 260 ms, degradando el throughput en un 30%.
3. **Hacer los agentes más rápidos no ayuda** si P10 es el cuello de botella. Este resultado es contraintuitivo pero emerge directamente del análisis de recursos.
4. **El modelo analítico predice con precisión el comportamiento empírico** (error < 0,3%), validando que el análisis de cuello de botella es una herramienta efectiva para este tipo de sistemas.

8 Verificación de invariantes

8.1 Verificación de invariantes de plaza

Los P-invariantes se verifican automáticamente tras **cada disparo** de la red, como parte del flujo de `fireTransition`. El método `RedPetri.verificarInvariantesPlaza()` evalúa las 6 ecuaciones sobre el marcado actual:

```
public void verificarInvariantesPlaza() {
    int[] m = marcado;
    if (m[1] + m[2] != 1)
        throw new ViolacionInvarianteException("PI-1: M(P1)+M(P2)=1", getMarcado());
    if (m[10] + m[11] + m[12] + m[13] != 1)
        throw new ViolacionInvarianteException("PI-2: ...", getMarcado());
    if (m[0]+m[2]+m[3]+m[5]+m[8]+m[9]+m[11]+m[12]+m[13]+m[14] != 5)
        throw new ViolacionInvarianteException("PI-3: ...", getMarcado());
    if (m[2] + m[3] + m[4] != 5)
        throw new ViolacionInvarianteException("PI-4: ...", getMarcado());
    if (m[5] + m[6] != 1)
        throw new ViolacionInvarianteException("PI-5: ...", getMarcado());
    if (m[7] + m[8] != 1)
        throw new ViolacionInvarianteException("PI-6: ...", getMarcado());
}
```

Si algún invariante resulta violado, se lanza una `ViolacionInvarianteException` (unchecked) que:

1. Atraviesa el `finally` del Monitor, liberando el lock correctamente.
2. Propaga hacia el `run()` del hilo, que no la captura.
3. Es interceptada por el `UncaughtExceptionHandler` registrado en Main, que imprime el diagnóstico y llama a `System.exit(1)`.

Una violación de invariante de plaza siempre indica un **bug en la implementación** de `RedPetri`, no un estado de carrera — es un fallo de lógica, no de concurrencia. Por ello la acción correcta es detener la ejecución inmediatamente con un mensaje descriptivo que incluye el marcado en el momento de la falla.

En las 51 ejecuciones realizadas (20 balanceadas + 31 priorizadas, 186 invariantes cada una = 9.486 disparos de T11 + disparos de H1-H5) no se detectó ninguna violación de P-invariante, lo que confirma la correcta implementación de las matrices Pre y Post.

8.2 Verificación de invariantes de transición por expresiones regulares

La verificación de T-invariantes se realiza mediante la clase `AnalizadorInvariantes`, que procesa el archivo de log al finalizar la ejecución.

8.2.1 Proceso de análisis

Paso 1 — Agrupación por cliente: el analizador recorre el log y agrupa las transiciones por ID de cliente. El resultado es un mapa `cliente_id -> [T0, T1, T3, ...]`:

```
[00:00:00.003] T0 (cliente=0)
[00:00:00.113] T1 (cliente=0)
[00:00:00.113] T3 (cliente=0)
[00:00:00.433] T4 (cliente=0)
[00:00:00.575] T7 (cliente=0)
[00:00:00.688] T8 (cliente=0)
[00:00:00.688] T11 (cliente=0)
==> cliente=0 : "T0 T1 T3 T4 T7 T8 T11"
```

Cada ID aparece exactamente en una secuencia sin interleaving con otros clientes (la agrupación por ID elimina el entrelazado natural del log).

Paso 2 — Clasificación por regex: cada secuencia se compara contra los 4 patrones exactos:

T-invariante	Expresión regular	Camino
I1	T0 T1 T3 T4 T7 T8 T11	Inferior + rechazado
I2	T0 T1 T3 T4 T6 T9 T10 T11	Inferior + aprobado
I3	T0 T1 T2 T5 T7 T8 T11	Superior + rechazado
I4	T0 T1 T2 T5 T6 T9 T10 T11	Superior + aprobado

Los patrones son exactos (sin `.*`): cada secuencia representa un único camino determinado y no hay ambigüedad posible. Se implementan con `Pattern.compile` de Java y se aplican con `Matcher.matches()`.

Paso 3 — Verificación: se clasifica cada secuencia en una de tres categorías:

- **Completa (I1–I4):** la secuencia termina en T11 y coincide con un patrón → se contabiliza.
- **Incompleta:** la secuencia no termina en T11 → cliente en vuelo al momento del shutdown → se descarta.
- **Inválida:** la secuencia termina en T11 pero no coincide con ningún patrón → indica un bug en la red → se reporta.

En ninguna de las 51 ejecuciones se encontró una secuencia inválida, lo que confirma que todos los caminos completados corresponden exactamente a uno de los 4 T-invariantes de la red.

8.2.2 Ejemplo de reporte generado

```
=====
ANÁLISIS DE T-INVARIANTES
=====
Invariantes completados : 186 / 186  [OK]
```

Desglose por camino:

I1 (inferior + rechazado) :	47	(25,3%)
I2 (inferior + aprobado) :	46	(24,7%)
I3 (superior + rechazado) :	46	(24,7%)
I4 (superior + aprobado) :	47	(25,3%)

Distribución de agentes:

Agente superior (I3+I4) :	93	(50,0%)
Agente inferior (I1+I2) :	93	(50,0%)

Distribución de decisiones:

Confirmadas (I2+I4) :	93	(50,0%)
Canceladas (I1+I3) :	93	(50,0%)

=====

9 Conclusiones

El presente trabajo implementó un sistema concurrente completo basado en una Red de Petri que modela una agencia de viajes, alcanzando todos los objetivos establecidos por la consigna.

Sobre el modelo formal: la red de Petri resultó ser acotada (sin desbordamiento de tokens), libre de deadlock (ciclos garantizados por T11) y no segura de forma intencional (P0, P3, P4 y P9 admiten múltiples tokens para el procesamiento paralelo de clientes). Los 6 P-invariantes y 4 T-invariantes identificados y demostrados formalmente constituyen las leyes de conservación del sistema.

Sobre la implementación: la arquitectura con un único `ReentrantLock` global y `Condition` por transición resultó correcta, eficiente y fácil de razonar. El Monitor expone exclusivamente `fireTransition(int)` como interfaz pública, manteniendo el encapsulamiento completo del estado de la red. La verificación automática de P-invariantes tras cada disparo permitió detectar inmediatamente cualquier bug en las matrices.

Sobre los hilos: la aplicación de los algoritmos 4.1, 4.2 y 4.3 del artículo de referencia determinó de forma rigurosa la cantidad y responsabilidad de los 6 hilos implementados. La decisión de reducir `S_salida` de 5 hilos (teóricos) a 1 hilo (implementado) está justificada por la naturaleza inmediata de T11, que elimina el paralelismo que motivaría múltiples hilos en un segmento temporizado.

Sobre las políticas: la política balanceada logró una distribución 50,0% / 50,0% exacta en agentes y decisiones, con una variabilidad de ± 1 por los clientes en vuelo. La política priorizada obtuvo resultados completamente deterministas (75,3% agente superior, 80,1% aprobaciones) gracias al bloqueo bidireccional, que elimina la dependencia del resultado respecto al scheduling del SO.

Sobre el análisis temporal: el análisis de cuello de botella identificó a P10 (agente aprobador) como el recurso limitante del sistema. La predicción analítica (37,2 s balanceada, 48,3 s priorizada) coincidió con los promedios empíricos (37,31 s y 48,44 s) con un error inferior al 0,3%, validando el modelo. La política priorizada es inherentemente un 30% más lenta que la balanceada debido al mayor tiempo de ocupación de P10 ocasionado por la mayor proporción de aprobaciones (que toman el camino más largo T9+T10 en lugar de T8).

10 Referencias

1. Ventre, G., & Micolini, O. (2022). *Algoritmos para determinar cantidad y responsabilidad de hilos en sistemas embebidos modelados con Redes de Petri S³PR*. FCEFyN — Universidad Nacional de Córdoba. Disponible en: <https://www.researchgate.net/publication/358104149>
2. Merlin, P. M. (1974). *A Study of the Recoverability of Computing Systems* (Tesis doctoral). University of California, Irvine.
3. PIPE — Platform Independent Petri net Editor. Herramienta de modelado y análisis de Redes de Petri. Disponible en: <https://pipe2.sourceforge.net/>
4. Oracle Corporation. (2024). *Java SE 21 API Specification — java.util.concurrent.locks*. Disponible en: <https://docs.oracle.com/en/java/docs/api/java.base/java/util/concurrent/locks/package-summary.html>
5. PlantUML. (2024). *Open-source tool for creating UML diagrams from plain text descriptions*. Disponible en: <https://plantuml.com/>